Projekt ZTPAI | Aleksandra Gołek | 151429

Aplikacja CoffeeBoard

1. Stack technologiczny

1.1 Backend (Spring Boot)

- **Java 21+**: Główny język programowania aplikacji serwerowej.
- **Spring Boot (3.4.2)**: Główny framework aplikacyjny przyspieszający konfigurację i development.
- **Spring Web**: Moduł odpowiedzialny za wystawienie interfejsu **REST API**, obsługę żądań HTTP oraz mapowanie danych do formatu JSON.
- **Spring Data JPA / Hibernate**: Warstwa dostępu do danych (ORM), pozwalająca na obiektowe mapowanie encji i łatwe zarządzanie operacjami CRUD.
- **Spring Security**: Moduł zapewniający bezpieczeństwo aplikacji (implementacja Basic Auth) oraz bezpieczne haszowanie haseł użytkowników za pomocą algorytmu **BCrypt**.
- **Maven**: Narzędzie do zarządzania zależnościami i cyklem budowania projektu.

1.2 Frontend (Angular)

- **Angular**: Framework do budowania dynamicznych aplikacji typu Single Page Application (SPA).
- **TypeScript**: Ściśle typowany język programowania, na którym opiera się logika frontendu.
- **RxJS**: Biblioteka wykorzystywana do programowania reaktywnego i asynchronicznej komunikacji z REST API (zapytania HTTP).
- **Bootstrap 5.3**: Biblioteka CSS użyta do stworzenia responsywnego, estetycznego interfejsu użytkownika (UI).
- **HTML5 / CSS3**: Standardowe technologie do budowy struktury i stylizacji (w tym customowa paleta kolorów dostosowana do motywu "TeamBoard Coffee").

1.3 Baza danych i Infrastruktura

- **PostgreSQL 16**: Relacyjna baza danych wykorzystywana do przechowywania informacji o użytkownikach i ogłoszeniach (zapisy na kawę).
- **Docker & Docker Compose**: Konteneryzacja środowiska bazodanowego, pozwalająca na szybkie, bezproblemowe uruchomienie bazy danych z pliku docker-compose.yaml.

2. Struktura projektu

```
ztpai/                              # Główny folder całego projektu
│
├── run-teamboard.ps1               # Skrypt automatyzujący uruchamianie bazy, backendu i frontendu
├── Regulamin_ZTPAI.pdf             # Wymagania projektowe
│
├── demo/                           # BACKEND (Spring Boot)
│   ├── docker-compose.yaml         # Konfiguracja kontenera z bazą danych PostgreSQL
│   ├── pom.xml                     # Konfiguracja zależności Mavena
│   ├── mvnw / mvnw.cmd             # Skrypty narzędzia Maven Wrapper
│   ├── .gitignore                  # Plik ignorowanych plików dla Gita (backend)
│   │
│   └── src/
│       ├── main/
│       │   ├── java/pl/edu/pk/demo/
│       │   │   ├── DemoApplication.java                # Główna klasa uruchomieniowa Spring Boot
│       │   │   │
│       │   │   ├── config/
│       │   │   │   └── SecurityConfig.java             # Konfiguracja Spring Security (CORS, Basic Auth)
│       │   │   │
│       │   │   ├── controller/                         # Warstwa kontrolerów (REST API)
│       │   │   │   ├── AdController.java               # Endpointy dla ogłoszeń (CRUD, dołączanie)
│       │   │   │   ├── AuthController.java             # Endpoint rejestracji użytkowników
│       │   │   │   └── SystemInfoController.java       # Endpointy informacyjne i testowe (powitania)
│       │   │   │
│       │   │   ├── dto/                                # Obiekty transferu danych (Data Transfer Objects)
│       │   │   │   ├── AdDTO.java
│       │   │   │   ├── AdRequestDTO.java               # DTO z walidacją dla nowych ogłoszeń
│       │   │   │   └── UserRegisterDTO.java            # DTO z walidacją rejestracji
│       │   │   │
│       │   │   ├── entity/                             # Modele bazy danych (JPA/Hibernate)
│       │   │   │   ├── Ad.java                         # Encja ogłoszenia (kawy)
│       │   │   │   └── User.java                       # Encja użytkownika
│       │   │   │
│       │   │   ├── event/                              # Architektura sterowana zdarzeniami (Events)
│       │   │   │   ├── NotificationListener.java       # Nasłuchiwacz zdarzeń
│       │   │   │   └── UserJoinedEvent.java            # Definicja zdarzenia dołączenia do kawy
│       │   │   │
│       │   │   ├── exception/                          # Globalna obsługa wyjątków
│       │   │   │   └── GlobalExceptionHandler.java     # Przechwytywanie błędów walidacji
│       │   │   │
│       │   │   ├── repository/                         # Warstwa dostępu do danych (Interfejsy bazodanowe)
│       │   │   │   ├── AdRepository.java
│       │   │   │   └── UserRepository.java
│       │   │   │
│       │   │   └── service/                            # Warstwa logiki biznesowej
│       │   │       └── AdService.java                  # Usługi zarządzające ogłoszeniami
│       │   │
│       │   └── resources/
│       │       └── application.properties              # Konfiguracja aplikacji (port, baza, Hibernate)
│       │
│       └── test/java/pl/edu/pk/demo/                   # TESTY JEDNOSTKOWE I INTEGRACYJNE
│           ├── DemoApplicationTests.java               # Test ładowania kontekstu Springa
│           └── AdServiceTest.java                      # Testy jednostkowe logiki biznesowej (AdService)
│
└── teamboard-frontend/             # FRONTEND (Angular)
    ├── src/
    │   ├── index.html              # Główny plik HTML aplikacji
    │   ├── main.ts                 # Punkt wejścia aplikacji Angular
    │   ├── styles.css              # Globalne style CSS
    │   │
    │   └── app/                    # Główny moduł i komponenty aplikacji
    │       ├── app.component.ts    # Logika głównego komponentu (TypeScript)
    │       ├── app.html            # Widok struktury frontendu (HTML)
    │       ├── app.component.css   # Style dedykowane dla głównego widoku
    │       ├── app.config.ts       # Konfiguracja aplikacji (np. provider HttpClient)
    │       ├── app.routes.ts       # Konfiguracja routingu (ścieżek)
    │       ├── app.spec.ts         # Testy automatyczne frontendu
    │       │
    │       └── services/
    │           └── ad.service.ts   # Serwis do komunikacji HTTP z backendem (REST API)
```

3. Lista wymagań i wykonanie

3.1 Wymagania na ocenę 3.0

<table>
<tr><th>Wymaganie</th><th>Wykonanie</th></tr>
<tr><td>Działający Spring Boot</td><td>Klasa DemoApplication.java z adnotacją @SpringBootApplication stanowi główny punkt wejścia aplikacji. Serwer poprawnie uruchamia się na porcie 8080.

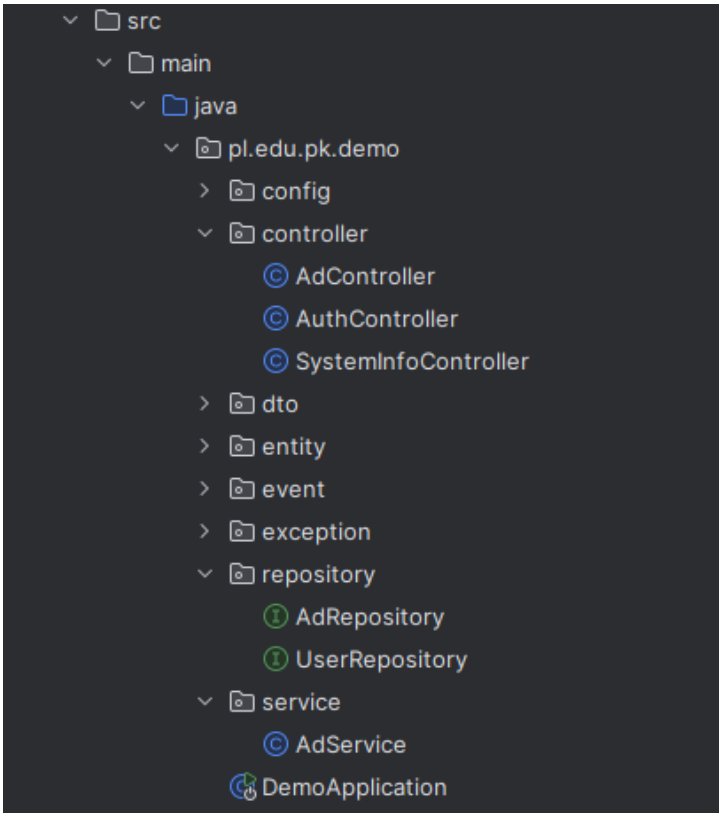
```java
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

    @Bean
    CommandLineRunner init(AdRepository adRepo, UserRepository userRepo, PasswordEncoder encoder) {
        return String[] args -> {

        };
    }
}
```
</td></tr>
<tr><td>Połączenie z bazą danych</td><td>Plik application.properties konfiguruje połączenie przez JDBC (spring.datasource.url=jdbc:postgresql://localhost:5432/coffee_db) do bazy PostgreSQL uruchamianej kontenerem z pliku docker-compose.yaml.

```properties
# Database Connection Settings
spring.datasource.url=jdbc:postgresql://localhost:5432/coffee_db
spring.datasource.username=user
spring.datasource.password=password
spring.datasource.driver-class-name=org.postgresql.Driver
spring.security.user.name=admin
spring.security.user.password=mojehaslo

# Hibernate Configuration
spring.jpa.hibernate.ddl-auto=update
#czyste logi essa
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

# Server Settings
server.port=8080
server.error.include-message=always
```
</td></tr>
</table>

CRUD dla 1 encji	Pełny CRUD (zrealizowany nawet dla dwóch encji: Ad i User). W AdController.java jest: • Create: @PostMapping createAd • Read: @GetMapping getAllAds • Update: @PatchMapping("/{id}/join") joinAd (aktualizuje listę uczestników) • Delete: @DeleteMapping("/{id}") deleteAd

```java
    @GetMapping
    public List<Ad> getAllAds() {
        return adService.getAllAds();
    }

    @PostMapping
    public Ad createAd(@Valid @RequestBody AdRequestDTO dto, Principal principal) {
        return adService.createAd(dto, principal.getName());
    }

    @PatchMapping("/{id}/join")
    public Ad joinAd(@PathVariable Long id, Principal principal) {
        Ad ad = repository.findById(id).orElseThrow(() -> new RuntimeException("Nie znaleziono!"));
        if (!ad.getParticipantNames().contains(principal.getName())) {
            ad.getParticipantNames().add(principal.getName());
            Ad savedAd = repository.save(ad);

            // KLUCZOWE: Publikujemy zdarzenie!
            eventPublisher.publishEvent(new UserJoinedEvent( source: this, ad.getTitle(), ad.getAuthor(), principal.getName()));

            return savedAd;
        }
        return ad;
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> deleteAd(@PathVariable Long id, Principal principal) {
        Ad ad = repository.findById(id).orElseThrow(() -> new RuntimeException("Nie znaleziono!"));
        if (ad.getAuthor().equals(principal.getName())) {
            repository.delete(ad);
            return ResponseEntity.ok().build();
        }
        return ResponseEntity.status(403).build();
    }
}
```

3.2 Wymagania na ocenę 4.0

Wymaganie	Wykonanie
Poprawna struktura warstwowa (Controller /Service/ Repository)	Gdzie to się znajduje: Pakietowa separacja warstw w pl.edu.pk.demo: • controller/ (np. AdController.java) obsługuje żądania HTTP. • service/ (np. AdService.java) zawiera logikę biznesową (np. bezpieczne przypisywanie autora ogłoszenia). • repository/ (np. AdRepository.java, UserRepository.java) odpowiada za interfejsy Spring Data JPA. 
DTO + Walidacja danych	Obiekty DTO w pakiecie dto/ (AdRequestDTO.java, UserRegisterDTO.java). Walidacja odbywa się za pomocą adnotacji jakarta.validation (np. @NotBlank, @Size) bezpośrednio w rekordach DTO. W kontrolerach dane wejściowe weryfikowane są klauzulą @Valid.

<table>
<tr>
<td></td>
<td>

```
dto
    AdDTO
    AdRequestDTO
    UserRegisterDTO
```

```java
package pl.edu.pk.demo.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record AdRequestDTO(  7 usages
        @NotBlank(message = "Tytuł nie może być pusty!")  2 usages
        @Size(min = 3, max = 50, message = "Tytuł musi mieć od 3 do 50 znaków")
        String title,

        @NotBlank(message = "Opis jest wymagany!")  2 usages
        @Size(min = 10, message = "Opis musi mieć min. 10 znaków")
        String description,

        @NotBlank(message = "Podaj miasto!")  2 usages
        String category
) {}
```

```java
public record UserRegisterDTO(  2 usages
        @NotBlank(message = "Login jest wymagany")  2 usages
        @Size(min = 3, message = "Login musi mieć min. 3 znaki")
        String username,

        @NotBlank(message = "Hasło jest wymagane")  1 usage
        @Size(min = 6, message = "Hasło musi mieć min. 6 znaków")
        String password
) {}
```

</td>
</tr>
<tr>
<td>

- Obsługa błędów

</td>
<td>

Klasa GlobalExceptionHandler.java w pakiecie exception/. Adnotacja @RestControllerAdvice przechwytuje wyjątki MethodArgumentNotValidException z całego systemu i mapuje je na estetyczne błędy w formacie JSON dla frontendu.

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach( FieldError error ->
                errors.put(error.getField(), error.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }
}
```

</td>
</tr>
</table>

Security (JWT lub Basic Auth)	Wdrożono Basic Auth. Plik SecurityConfig.java rezyduje w warstwie zabezpieczeń, udostępniając własnego UserDetailsService, który pobiera użytkowników prosto z bazy przy użyciu UserRepository. Hasła chronione są metodą szyfrującą BCryptPasswordEncoder.

```
@Bean
public UserDetailsService userDetailsService(UserRepository userRepository) {
    // Ta metoda mówi Springowi, jak znaleźć użytkownika w Twojej bazie danych
    return String username -> userRepository.findByUsername(username) Optional<User>
            .map( User u -> org.springframework.security.core.userdetails.User
                    .withUsername(u.getUsername())
                    .password(u.getPassword()) // Spring sam porówna to z hasłem z bazy używając BCrypt
                    .roles("USER")
                    .build()) Optional<UserDetails>
            .orElseThrow(() -> new UsernameNotFoundException("Nie znaleziono użytkownika: " + username));
}
```

3.3 Wymagania na ocenę 5.0

Wymaganie	Wykonanie
Unit Testy	Klasa testowa AdServiceTest.java (zlokalizowana w src/test/java). Wykorzystuje Mockito (adnotacje @Mock, @InjectMocks) do izolowanego przetestowania metod serwisu, takich jak weryfikacja czy podczas tworzenia ogłoszenia poprawnie przypisywany jest z sesji autor (np. logikę zapisu chronioną testem shouldCreateAdWithCorrectAuthor).

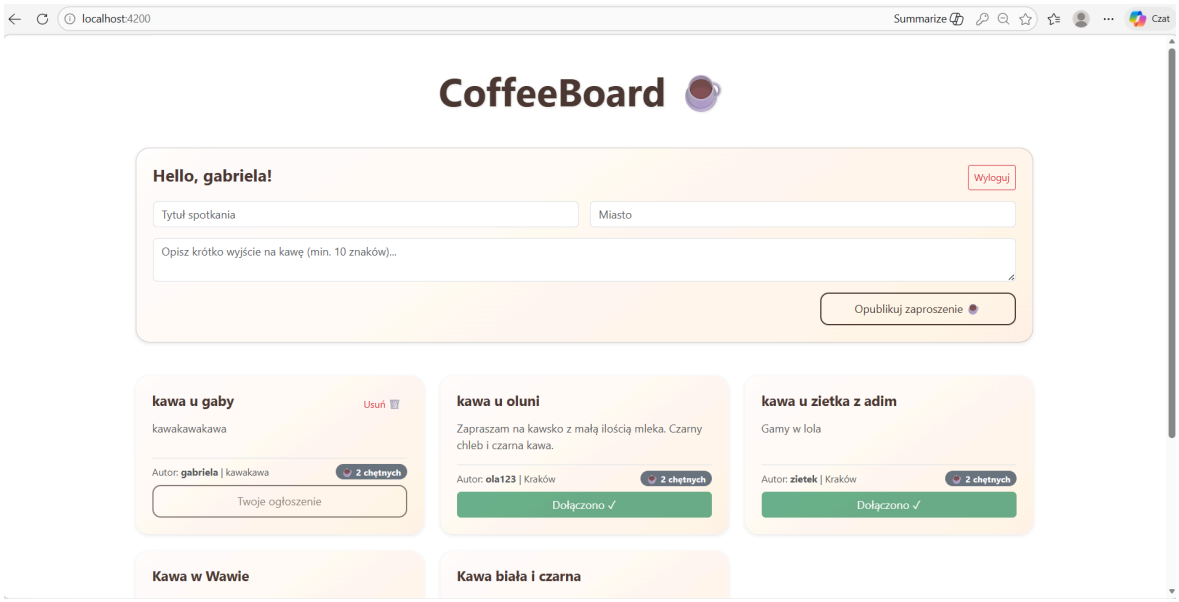
```
@Test
void shouldReturnAllAds() {
    // Given (Zakładamy, że baza zwraca jedną listę)
    when(adRepository.findAll()).thenReturn(Arrays.asList(testAd));

    // When (Wykonujemy logikę serwisu)
    List<Ad> ads = adService.getAllAds();

    // Then (Sprawdzamy, czy wynik jest poprawny)
    assertEquals( expected: 1, ads.size(), message: "Powinno zwrócić jedno ogłoszenie");
    assertEquals( expected: "Kawa w Krakowie", ads.get(0).getTitle());

    // Weryfikacja, czy serwis faktycznie odpytał repozytorium
    verify(adRepository).findAll();
}
```

```java
@Test
void shouldCreateAdWithCorrectAuthor() {
    // Given
    String currentLoggedUser = "admin_user";

    // Symulacja zapisu do bazy - zwracamy to, co weszło + nadajemy ID
    when(adRepository.save(any(Ad.class))).thenAnswer( InvocationOnMock invocation -> {
        Ad savedAd = invocation.getArgument( i: 0);
        savedAd.setId(99L);
        return savedAd;
    });

    // When (Próba utworzenia ogłoszenia przez serwis)
    Ad result = adService.createAd(testDto, currentLoggedUser);

    // Then (Sprawdzenie, czy serwis dobrze zmapował obiekt)
    assertNotNull(result);
    assertEquals( expected: 99L, result.getId(),  message: "Ogłoszenie powinno dostać ID po zapisie");
    assertEquals( expected: "Kawa w Krakowie", result.getTitle());

    // Najważniejszy test bezpieczeństwa: czy autor został nadpisany z sesji?
    assertEquals( expected: "admin_user", result.getAuthor(),  message: "Autor musi zgadzać się z przekazanym loginem z sesji");

    // Sprawdzenie czy kolekcja uczestników nie jest nullem (ochrona przed błędem 500)
    assertNotNull(result.getParticipantNames(),  message: "Lista uczestników nie może być nullem");
    assertTrue(result.getParticipantNames().isEmpty(),  message: "Nowe ogłoszenie nie powinno mieć uczestników");

    // Weryfikacja, czy metoda save została wywołana dokładnie raz
    verify(adRepository).save(any(Ad.class));
}
```

Events LUB Kolejki

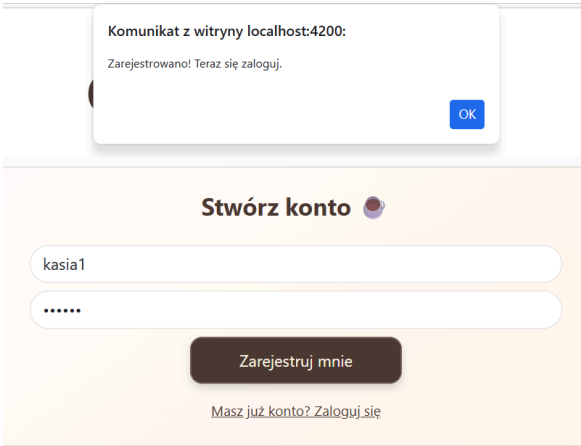
Użyto mechanizmu zdarzeń (Events) wbudowanego w Springa. Akcja dołączenia na kawę (AdController.java -> joinAd) wysyła w system zdarzenie poprzez eventPublisher.publishEvent(...). Wydarzenie jest ustrukturyzowane w UserJoinedEvent.java, po czym asynchronicznie wyłapywane przez nasłuchiwacz NotificationListener.java (@EventListener), logujący ten incydent w konsoli.

```java
@Component
public class NotificationListener {

    // Profesjonalny logger wbudowany w Springa
    private static final Logger logger = LoggerFactory.getLogger(NotificationListener.class);  1 usage

    @EventListener
    public void handleUserJoinedEvent(UserJoinedEvent event) {
        // Tekst bez polskich znaków i emoji - w 100% bezpieczny dla każdej konsoli
        logger.info("[SYSTEM POWIADOMIEN]: Uzytkownik '{}' wpadnie na kawe '{}' (Autor: {})",
                event.getParticipant(), event.getAdTitle(), event.getAuthor());
    }
}
```

Czysty kod	Kod cechuje się angielskim nazewnictwem metod i klas, drobnymi klasami, wstrzykiwaniem zależności przez konstruktory (np. w AdController.java) oraz brakiem nieużywanych zmiennych. Logika biznesowa nie przenika do kontrolerów.
Frontend (prosta aplikacja Angular konsumująca API)	Pełnoprawna aplikacja SPA w Angularze (teamboard-frontend), komunikująca się asynchronicznie przez HttpClient w serwisie ad.service.ts i wyświetlająca komponenty na podstawie reaktywnego odświeżania bindowanych modeli (app.component.ts). 

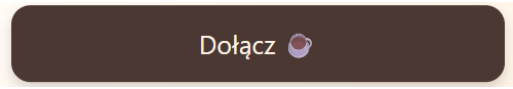
3.4 Prezentacja warstwy frontendowej

Rejestracja

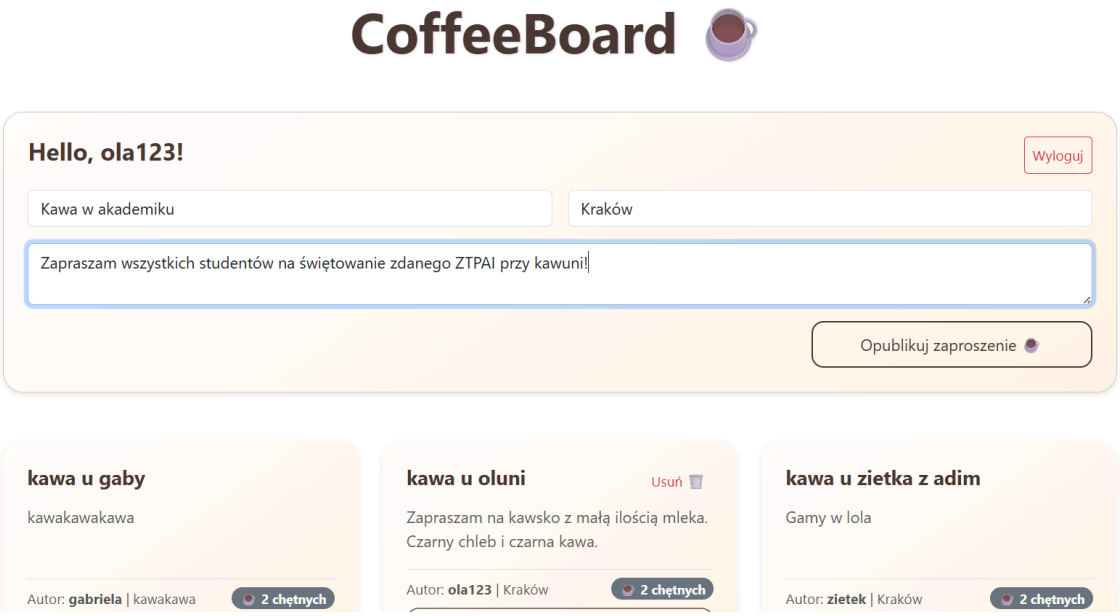


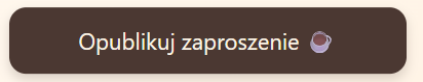
Logowanie

Zapisanie się na wydarzenie

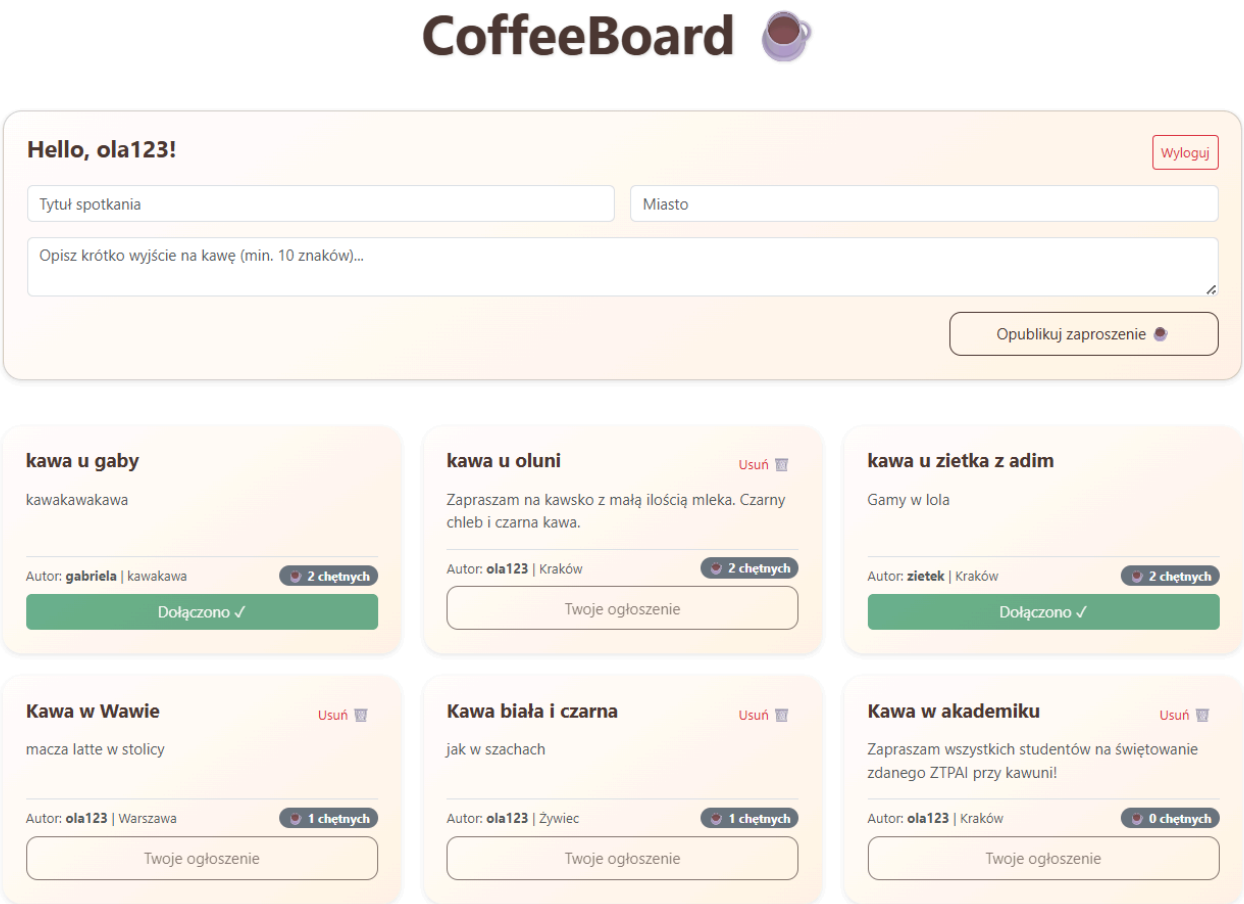
Klikamy: 

Tworzenie ogłoszenia

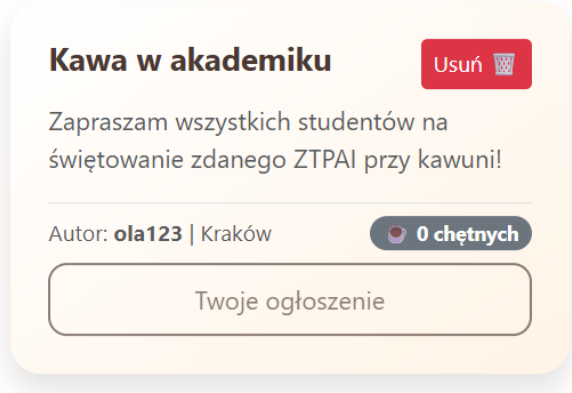


Klikamy: 

Przeglądanie ogłoszeń (dodane ogłoszenia pojawiają się na liście w czasie rzeczywistym)



Usuwanie ogłoszenia



4. Warstwa Biznesowa - Specyfika dla aplikacji "CoffeeBoard"

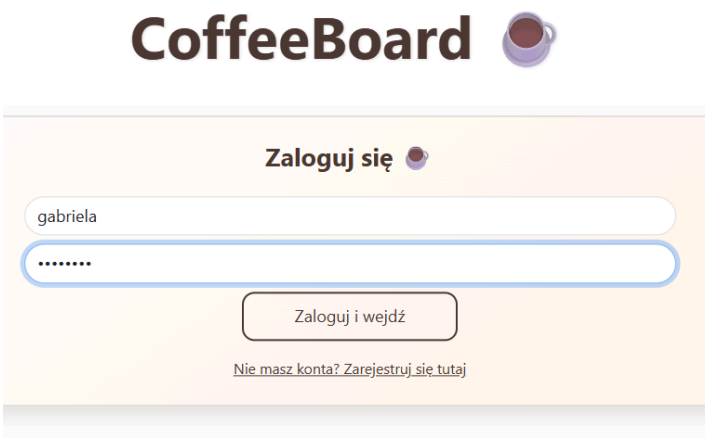
Ta prosta aplikacja ma na celu stworzenie przestrzeni dla ludzi, którzy chcą spotkać się na kawę. Zalogowani użytkownicy mogą tworzyć ogłoszenia i zapisywać się na nie. Niezalogowany użytkownik może stworzyć bezpiecznie konto w systemie.

- Konta i spersonalizowane powitania: Po udanym logowaniu Angular odpytuje unikalny endpoint pobierając powitanie, co realizowane jest kodem w SystemInfoController.java jako metoda checkUserStatus(@PathVariable String name), witająca zwrotnie np. "Hello, gabriela!".

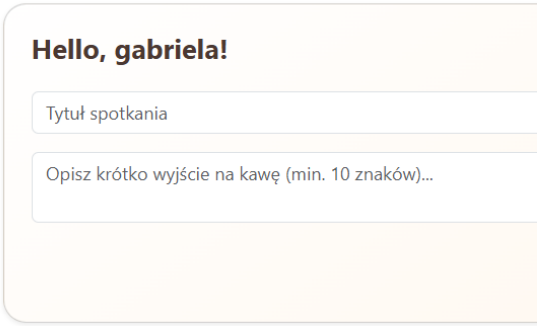
```
@GetMapping("/hello/{name}")
public String checkUserStatus(@PathVariable String name) {
    return "Hello, " + name + "!"; // ma zwracac na przyklad. "Hello, gabriela!"
}
```

```
<div *ngIf="currentUser" class="card coffee-card mb-5 p-4 shadow-sm">
  <div class="d-flex justify-content-between mb-3">
    <h4 class="coffee-text fw-bold">{{ welcomeMessage }}</h4>
    <button (click)="logout()" class="btn btn-sm btn-outline-danger">Wyloguj</button>
  </div>
```

Fragmenty kodu



Proces logowania użytkownika



Spersonalizowane powitanie

- Dołączenie tylko raz / Inkrementacja licznika: W klasie Ad.java trzymamy uczestników jako unikalną kolekcję @ElementCollection Set<String> participantNames. Ponadto w endpointcie dołączania AdController.java widnieje rygorystyczny warunek if (!ad.getParticipantNames().contains(principal.getName())), po czym dodajemy chętnego do Setu i zapisujemy bazę. Licznik aktualizuje się na żywo w HTML-u dzięki bindowaniu Angulara: ad.participantNames?.length.

```
@ElementCollection(fetch = FetchType.EAGER)  2 usages
private Set<String> participantNames = new HashSet<>();
```

```
<span class="badge bg-secondary rounded-pill" style="...">
  {{ ad.participantNames?.length || 0 }} chętnych
</span>
```

```java
@PatchMapping("/{id}/join")
public Ad joinAd(@PathVariable Long id, Principal principal) {
    Ad ad = repository.findById(id).orElseThrow(() -> new RuntimeException("Nie znaleziono!"));
    if (!ad.getParticipantNames().contains(principal.getName())) {
        ad.getParticipantNames().add(principal.getName());
        Ad savedAd = repository.save(ad);

        // KLUCZOWE: Publikujemy zdarzenie!
        eventPublisher.publishEvent(new UserJoinedEvent( source: this, ad.getTitle(), ad.getAuthor(), principal.getName()));

        return savedAd;
    }
    return ad;
}
```

Fragmenty kodu

Widok po stronie użytkownika (inkrement i ilosć chętnych)

5. Uruchomienie

5.1 Wymagania wstępne

Aby poprawnie uruchomić projekt na lokalnej maszynie, wymagane jest zainstalowanie:

- **Java** (wersja 21 lub nowsza)
- **Node.js** oraz **npm** (do uruchomienia frontendu w Angularze)
- **Docker** (do postawienia bazy danych PostgreSQL)

5. 2 Instrukcja uruchomienia - Opcja 1: Szybka - Windows PowerShell

Projekt zawiera skrypt automatyzujący start całego środowiska.

1. Uruchom terminal PowerShell w głównym folderze projektu (ztpai).

2. Wpisz komendę: .\run-teamboard.ps1 (Uwaga: Może być wymagane odblokowanie skryptów komendą: powershell -ExecutionPolicy Bypass -File .\run-teamboard.ps1)

5.3 Instrukcja uruchomienia - Opcja 2: Ręczna

1. Baza danych: Wejdź do folderu demo i wpisz w terminalu: docker-compose up -d. Baza wstanie w tle na porcie 5432.
2. Backend: Będąc wciąż w folderze demo, wpisz: .\mvnw spring-boot:run (lub ./mvnw spring-boot:run na Mac/Linux). API uruchomi się pod http://localhost:8080.
3. Frontend: Otwórz nowy terminal, wejdź do folderu teamboard-frontend, zainstaluj paczki komendą npm install, a następnie uruchom aplikację komendą npm start (lub ng serve). Frontend będzie dostępny pod **http://localhost:4200.**

5.4 Dane testowe i logowanie

Po wejściu na stronę http://localhost:4200 można założyć nowe konto użytkownika, używając formularza (min. 3 znaki dla loginu, min. 6 dla hasła) lub zalogować się na domyślne konto administratora zaszyte w konfiguracji:

- Login: admin
- Hasło: mojehaslo